

TP 6 React Native :

Dans le TP 5 :

- Les tâches sont stockées avec `useState` dans `TodoListScreen`
- Impossible de partager/modifier les tâches depuis un autre écran

👉 **Problème :**

Comment partager la liste des tâches entre plusieurs écrans sans passer par les props ?

Installer React Navigation et dépendances

```
npm install @reduxjs/toolkit react-redux
```

Création du store Redux

/store/store.js

```
import { configureStore } from "@reduxjs/toolkit";
import todosReducer from "../todosSlice";

export const store = configureStore({
  reducer: {
    todos: todosReducer,
  },
});
```

Création du reducer

/store/todosSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const todosSlice = createSlice({
  name: "todos",
  initialState: [],
  reducers: {
    addTodo: (state, action) => {
      state.push(action.payload);
    },
    removeTodo: (state, action) => {
```

```

        return state.filter((t) => t.id !== action.payload);
    },
  },
});

export const { addTodo, removeTodo } = todosSlice.actions;
export default todosSlice.reducer;

```

Fournir le store à l'application

App.js

```

import { NavigationContainer } from "@react-navigation/native";
import AuthProvider, { AuthContext } from "../context/AuthContext";
import AppDrawer from "../navigation/AppDrawer";
import LoginScreen from "../screens/LoginScreen";
import { useContext } from "react";

import { Provider } from "react-redux";
import { store } from "../store/store";

function RootNavigator() {
  const { user } = useContext(AuthContext);

  return user ? <AppDrawer /> : <LoginScreen />;
}

export default function App() {
  return (
    <Provider store={store}>
      <AuthProvider>
        <NavigationContainer>
          <RootNavigator />
        </NavigationContainer>
      </AuthProvider>
    </Provider>
  );
}

```

Utilisation dans TodoListScreen

/screens/TodoListScreen.js

```
import { View, Text, FlatList, TouchableOpacity } from "react-native";
import { useSelector, useDispatch } from "react-redux";
import { useEffect } from "react";
import { addTodo } from "../store/todosSlice";
import AppBar from "../components/AppBar";

export default function TodoListScreen({ navigation }) {
  const todos = useSelector(state => state.todos);
  const dispatch = useDispatch();

  useEffect(() => {
    dispatch(addTodo({ id: 1, title: "Faire les courses" }));
    dispatch(addTodo({ id: 2, title: "Sortir le chien" }));
    dispatch(addTodo({ id: 3, title: "Coder une app RN" }));
  }, []);

  return (
    <View style={{ flex: 1, padding: 20 }}>
      <AppBar title="Mes tâches" />

      <FlatList
        data={todos}
        keyExtractor={(i) => i.id.toString()}
        renderItem={({ item }) => (
          <TouchableOpacity
            onPress={() =>
              navigation.navigate("Détails", item)
            }
          >
            <Text style={{ padding: 10, fontSize: 18 }}>
              {item.title}
            </Text>
          </TouchableOpacity>
        )}
      />
    </View>
  );
}
```

Suppression depuis `TodoDetailsScreen`

```
import { View, Text, Button } from "react-native";
import { useDispatch } from "react-redux";
import { removeTodo } from "../store/todosSlice";

export default function TodoDetailsScreen({ route, navigation }) {
  const { id, title } = route.params;
  const dispatch = useDispatch();

  const handleDelete = () => {
    dispatch(removeTodo(id));
    navigation.goBack();
  };

  return (
    <View style={{ flex: 1, padding: 20 }}>
      <Text style={{ fontSize: 24 }}>{title}</Text>

      <Button
        title="Supprimer cette tâche"
        color="red"
        onPress={handleDelete}
      />
    </View>
  );
}
```

Gestion d'état globale avec ZUSTAND

Installation: `npm install zustand`

Création du store Zustand

```
/store/useTodoStore.js
import { create } from "zustand";

export const useTodoStore = create((set) => ({
  // état global
  todos: [],

  // actions
  addTodo: (todo) =>
    set((state) => ({
      todos: [...state.todos, todo],
    })),
})),
```

```
removeTodo: (id) =>
  set((state) => ({
    todos: state.todos.filter((t) => t.id !== id),
  })),
}));
```

Utilisation dans TodoListScreen

```
import { useTodoStore } from "../store/useTodoStore";

const { todos, addTodo } = useTodoStore();

useEffect(() => {

  addTodo({ id: 1, title: "Faire les courses" });

  addTodo({ id: 2, title: "Sortir le chien" });

  addTodo({ id: 3, title: "Coder une app RN" });

}, []);
```

Exercice supplémentaire :

Dans `TodoDetailsScreen` :

1. Récupérer l'action de suppression depuis le store Zustand
2. Implémenter un bouton « **Supprimer** »
3. Après la suppression :
 - revenir à l'écran précédent
 - constater l'impact sur la liste